Indicators of Heavy Traffic on I-94

By Jeseena Parveen K https://www.linkedin.com/in/jeseena-parveen-k/

Dataset: https://archive.ics.uci.edu/dataset/492/metro+interstate+traffic+volume

Goal: Determine a few indicators of heavy traffic on 1-94.

These indicators can be weather type, time of the day, time of the week, etc. o i.e., we may find out that the traffic is usually heavier in the summer or when it snows.

In [44]:
```
%pip install numpy pandas matplotlib seaborn
```

```
Requirement already satisfied: numpy in c:\users\jesee\appdata\local\programs\python
\python312\lib\site-packages (2.1.2)Note: you may need to restart the kernel to use
updated packages.
[notice] A new release of pip is available: 25.2 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
Requirement already satisfied: pandas in c:\users\jesee\appdata\local\programs\pytho
n\python312\lib\site-packages (2.2.3)
Requirement already satisfied: matplotlib in c:\users\jesee\appdata\local\programs\p
ython\python312\lib\site-packages (3.9.2)
Requirement already satisfied: seaborn in c:\users\jesee\appdata\local\programs\pyth
on\python312\lib\site-packages (0.13.2)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\jesee\appdata\loca
l\programs\python\python312\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\jesee\appdata\local\programs
\python\python312\lib\site-packages (from pandas) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\jesee\appdata\local\progra
ms\python\python312\lib\site-packages (from pandas) (2024.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\jesee\appdata\local\prog
rams\python\python312\lib\site-packages (from matplotlib) (1.3.0)
Requirement already satisfied: cycler>=0.10 in c:\users\jesee\appdata\local\programs
\python\python312\lib\site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\jesee\appdata\local\pro
grams\python\python312\lib\site-packages (from matplotlib) (4.54.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\jesee\appdata\local\pro
grams\python\python312\lib\site-packages (from matplotlib) (1.4.7)
Requirement already satisfied: packaging>=20.0 in c:\users\jesee\appdata\local\progr
ams\python\python312\lib\site-packages (from matplotlib) (24.1)
Requirement already satisfied: pillow>=8 in c:\users\jesee\appdata\local\programs\py
thon\python312\lib\site-packages (from matplotlib) (11.0.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\jesee\appdata\local\prog
rams\python\python312\lib\site-packages (from matplotlib) (3.2.0)
Requirement already satisfied: six>=1.5 in c:\users\jesee\appdata\local\programs\pyt
hon\python312\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.16.0)
```

In [45]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
%matplotlib inline
import os
```

EDA

In [46]:
```
traffic_i94 = pd.read_csv(r"C:/Users/jesee/Downloads/Python Projects/metro+intersta
traffic_i94.head()
```

Out[46]:

	holiday	temp	rain_1h	snow_1h	clouds_all	weather_main	weather_description	date_
0	NaN	288.28	0.0	0.0	40	Clouds	scattered clouds	2012 09:(
1	NaN	289.36	0.0	0.0	75	Clouds	broken clouds	2012 10:(
2	NaN	289.58	0.0	0.0	90	Clouds	overcast clouds	2012 11:(
3	NaN	290.13	0.0	0.0	90	Clouds	overcast clouds	2012 12:(
4	NaN	291.14	0.0	0.0	75	Clouds	broken clouds	2012 13:(

In [47]:
```
traffic_i94.tail(5)
```

Out[47]:

	holiday	temp	rain_1h	snow_1h	clouds_all	weather_main	weather_description
48199	NaN	283.45	0.0	0.0	75	Clouds	broken clouds
48200	NaN	282.76	0.0	0.0	90	Clouds	overcast clouds
48201	NaN	282.73	0.0	0.0	90	Thunderstorm	proximity thunderstorm
48202	NaN	282.09	0.0	0.0	90	Clouds	overcast clouds
48203	NaN	282.12	0.0	0.0	90	Clouds	overcast clouds

In [48]:
```
print("Size of data:", traffic_i94.shape) #find no.of rows & columns
```

```
Size of data: (48204, 9)
```

In [49]:
```
print("Info of data:", traffic_i94.info()) #find datatypes & null values
# 61 non-nulll values in holiday, only done on midnight in the corresponding holida
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48204 entries, 0 to 48203
Data columns (total 9 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   holiday              61 non-null     object
 1   temp                 48204 non-null  float64
 2   rain_1h              48204 non-null  float64
 3   snow_1h              48204 non-null  float64
 4   clouds_all           48204 non-null  int64
 5   weather_main         48204 non-null  object
 6   weather_description  48204 non-null  object
 7   date_time            48204 non-null  object
 8   traffic_volume       48204 non-null  int64
dtypes: float64(3), int64(2), object(4)
memory usage: 3.3+ MB
Info of data: None
```

In [50]:
```
traffic_i94.columns #find the columns
```

Out[50]:
```
Index(['holiday', 'temp', 'rain_1h', 'snow_1h', 'clouds_all', 'weather_main',
       'weather_description', 'date_time', 'traffic_volume'],
      dtype='object')
```

In [51]:
```
traffic_i94.columns.to_list() #find the columns list-wise
```

Out[51]:
```
['holiday',
 'temp',
 'rain_1h',
 'snow_1h',
 'clouds_all',
 'weather_main',
 'weather_description',
 'date_time',
 'traffic_volume']
```

In [52]:
```
traffic_i94.dtypes
```

Out[52]:
```
holiday                 object
temp                   float64
rain_1h                float64
snow_1h                float64
clouds_all               int64
weather_main            object
weather_description     object
date_time               object
traffic_volume           int64
dtype: object
```

In [53]:
```
traffic_i94.count()
```

Out[53]:
```
holiday                   61
temp                   48204
rain_1h                48204
snow_1h                48204
clouds_all             48204
weather_main           48204
weather_description    48204
date_time              48204
traffic_volume         48204
dtype: int64
```

In [54]:
```
traffic_i94.isnull()
```

Out[54]:

	holiday	temp	rain_1h	snow_1h	clouds_all	weather_main	weather_description	d
0	True	False	False	False	False	False	False	
1	True	False	False	False	False	False	False	
2	True	False	False	False	False	False	False	
3	True	False	False	False	False	False	False	
4	True	False	False	False	False	False	False	
...	...	...	...	...	...	...	...	
48199	True	False	False	False	False	False	False	
48200	True	False	False	False	False	False	False	
48201	True	False	False	False	False	False	False	
48202	True	False	False	False	False	False	False	
48203	True	False	False	False	False	False	False	

48204 rows × 9 columns

In [55]:
```
traffic_i94.isnull().sum()
```

Out[55]:
```
holiday                48143
temp                       0
rain_1h                    0
snow_1h                    0
clouds_all                 0
weather_main               0
weather_description        0
date_time                  0
traffic_volume             0
dtype: int64
```

In [56]:
```
traffic_i94['holiday'].value_counts() # counts how many times each unique value app
```

Out[56]:
```
holiday
Labor Day                   7
Christmas Day               6
Thanksgiving Day            6
Martin Luther King Jr Day   6
New Years Day               6
Veterans Day                5
Columbus Day                5
Memorial Day                5
Washingtons Birthday        5
State Fair                  5
Independence Day            5
Name: count, dtype: int64
```

In [57]:
```
# Filtering rows where the column is NOT null
non_null_values_traffic_i94 = traffic_i94[traffic_i94['holiday'].notnull()]['holida

# Display the result
print(non_null_values_traffic_i94)
```

```
['Columbus Day', 'Veterans Day', 'Thanksgiving Day', 'Christmas Day', 'New Years Da
y', 'Washingtons Birthday', 'Memorial Day', 'Independence Day', 'State Fair', 'Labor
Day', 'Columbus Day', 'Veterans Day', 'Thanksgiving Day', 'Christmas Day', 'New Year
s Day', 'Martin Luther King Jr Day', 'Washingtons Birthday', 'Memorial Day', 'Indepe
ndence Day', 'State Fair', 'Labor Day', 'Columbus Day', 'Veterans Day', 'Thanksgivin
g Day', 'Thanksgiving Day', 'Christmas Day', 'New Years Day', 'New Years Day', 'Wash
ingtons Birthday', 'Memorial Day', 'Independence Day', 'State Fair', 'Labor Day', 'L
abor Day', 'Columbus Day', 'Veterans Day', 'Thanksgiving Day', 'Christmas Day', 'Chr
istmas Day', 'New Years Day', 'Martin Luther King Jr Day', 'Martin Luther King Jr Da
y', 'Washingtons Birthday', 'Memorial Day', 'Independence Day', 'State Fair', 'Labor
Day', 'Columbus Day', 'Veterans Day', 'Thanksgiving Day', 'Christmas Day', 'New Year
s Day', 'Martin Luther King Jr Day', 'Martin Luther King Jr Day', 'Martin Luther Kin
g Jr Day', 'Washingtons Birthday', 'Memorial Day', 'Independence Day', 'State Fair',
'Labor Day', 'Labor Day']
```

In [58]:
```
traffic_i94.describe() #Statistical report of the dataset
```

Out[58]:

	temp	rain_1h	snow_1h	clouds_all	traffic_volume
count	48204.000000	48204.000000	48204.000000	48204.000000	48204.000000
mean	281.205870	0.334264	0.000222	49.362231	3259.818355
std	13.338232	44.789133	0.008168	39.015750	1986.860670
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	272.160000	0.000000	0.000000	1.000000	1193.000000
50%	282.450000	0.000000	0.000000	64.000000	3380.000000
75%	291.806000	0.000000	0.000000	90.000000	4933.000000
max	310.070000	9831.300000	0.510000	100.000000	7280.000000

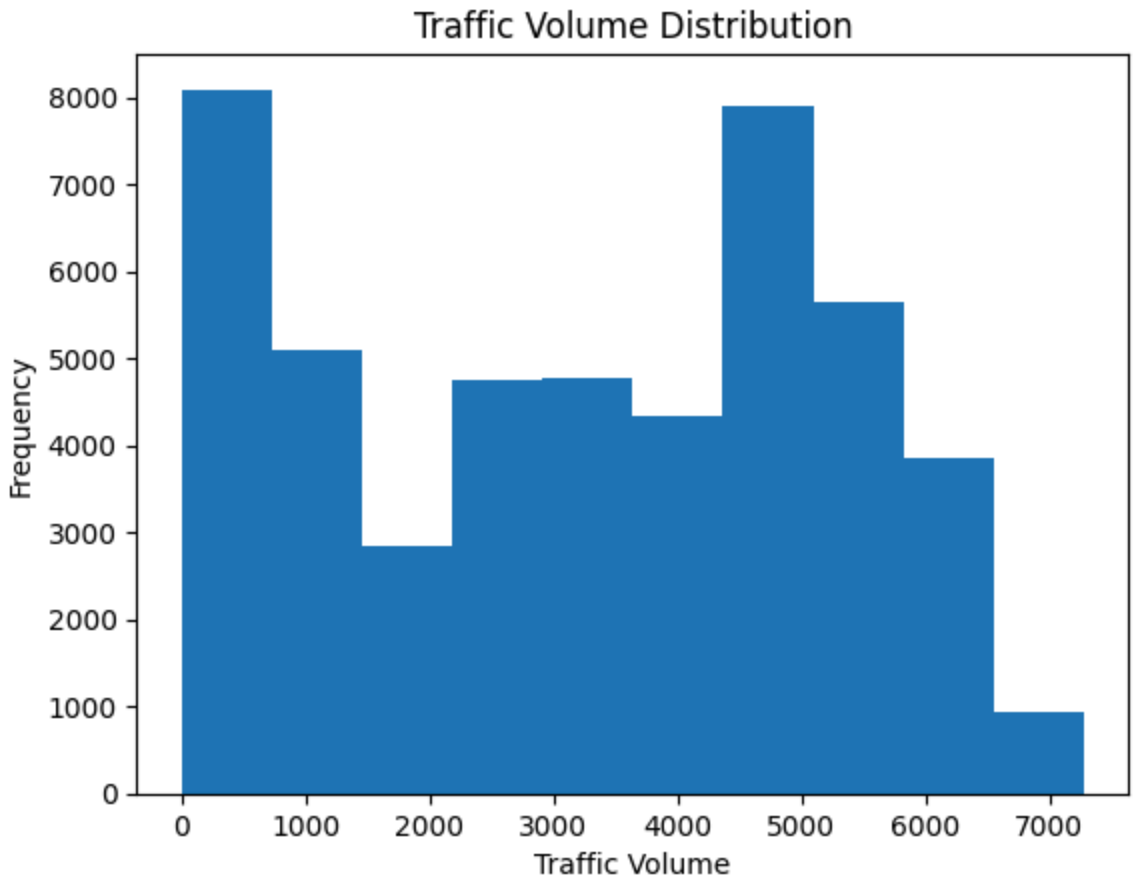
In [59]:
```
# traffic_i94.describe(include='all')
```

The dataset has 48,204 rows and 9 columns, and there are no null values. Each row describes traffic and weather data for a specific hour. we have data from 2012-10-02 09:00:00 until 2018-09-30 23:00:00.

Distribution of traffic volume

In [152…
```
# distribution of traffic volume column
plt.hist(traffic_i94['traffic_volume'])
plt.xlabel('Traffic Volume')
plt.ylabel('Frequency')
```

```
plt.title('Traffic Volume Distribution')
plt.show()
```



Between 2012-10-02 09:00:00 and 2018-09-30 23:00:00, the hourly traffic volume varied from 0 to 7,280 cars, with an average of 3,260 cars.

About 25% of the time, there were only 1,193 cars or fewer passing the station each hour this probably occurs during the night, or when a road is under construction. However, about 25% of the time, the traffic volume was four times as much (4,933 cars or more).

This observation gives our analysis an interesting direction: comparing daytime data with nighttime data.

In [61]:
```
# Analyze between Day & Night time
traffic_i94['date_time'] = pd.to_datetime(traffic_i94['date_time'])

day = traffic_i94.copy()[(traffic_i94['date_time'].dt.hour >= 7) & (traffic_i94['da
print(day.shape)

night = traffic_i94.copy()[(traffic_i94['date_time'].dt.hour >= 19) | (traffic_i94[
print(night.shape)
```

```
(23877, 9)
(24327, 9)
```

In [62]:
```
traffic_i94.iloc[176:178]
```

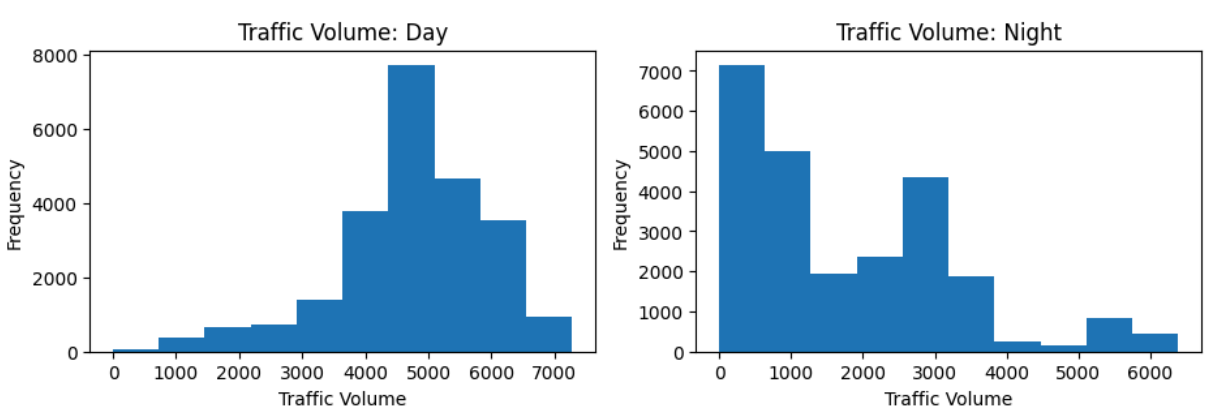
Out[62]:

	holiday	temp	rain_1h	snow_1h	clouds_all	weather_main	weather_description	dat
176	NaN	281.17	0.0	0.0	90	Clouds	overcast clouds	2(0
177	NaN	281.25	0.0	0.0	92	Clear	sky is clear	2(0

This significant difference in row numbers between day and night is due to few hours of missing data. For instance, if you look at rows 176 and 177, you'll notice there's no data for two hours (4 and 5).

In [68]:

```
# sub-plots helps put lof of graphs in custom fomratting in output
plt.figure(figsize=(11,3))

plt.subplot(1, 2, 1)
plt.hist(day['traffic_volume'])
plt.title('Traffic Volume: Day')
plt.ylabel('Frequency')
plt.xlabel('Traffic Volume')
plt.xlim(-100, 7500)
plt.ylim(0, 8000)


plt.subplot(1, 2, 2)
plt.hist(night['traffic_volume'])
plt.title('Traffic Volume: Night')
plt.ylabel('Frequency')
plt.xlabel('Traffic Volume')
plt.xlim(-100, 7500)
plt.ylim(0, 8000)


plt.show()
```



In [69]:

```
day['traffic_volume'].describe()
```

Out[69]:
```
count    23877.000000
mean      4762.047452
std       1174.546482
min          0.000000
25%       4252.000000
50%       4820.000000
75%       5559.000000
max       7280.000000
Name: traffic_volume, dtype: float64
```

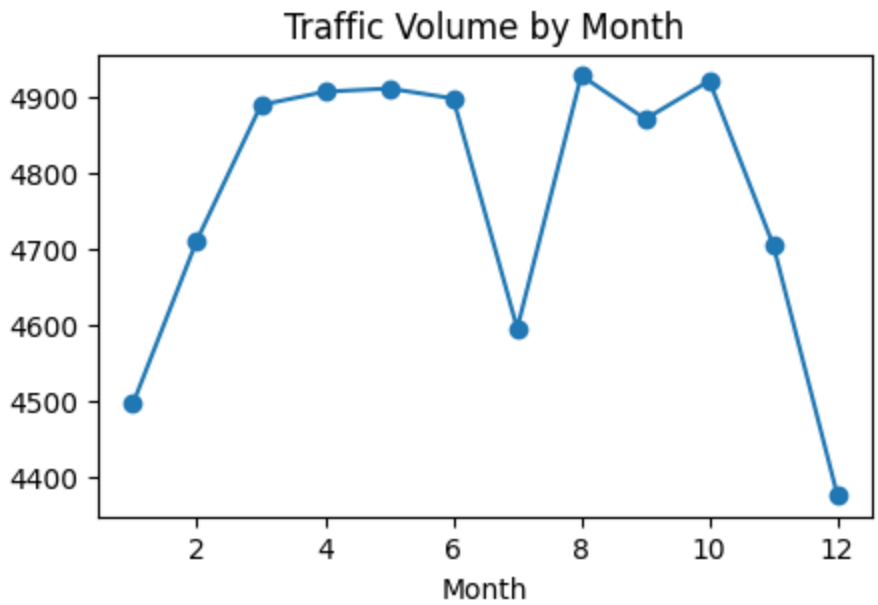
In [70]:
```
night['traffic_volume'].describe()
```

Out[70]:
```
count    24327.000000
mean      1785.377441
std       1441.951197
min          0.000000
25%        530.000000
50%       1287.000000
75%       2819.000000
max       6386.000000
Name: traffic_volume, dtype: float64
```

Daytime traffic is left-skewed, meaning traffic volume is generally high, with 75% of observations exceeding 4,252 cars per hour. In contrast, nighttime traffic is right-skewed, indicating mostly low traffic, with 75% of values below 2,819 cars per hour. Since heavy traffic occurs primarily during the day, further analysis will focus on daytime data.

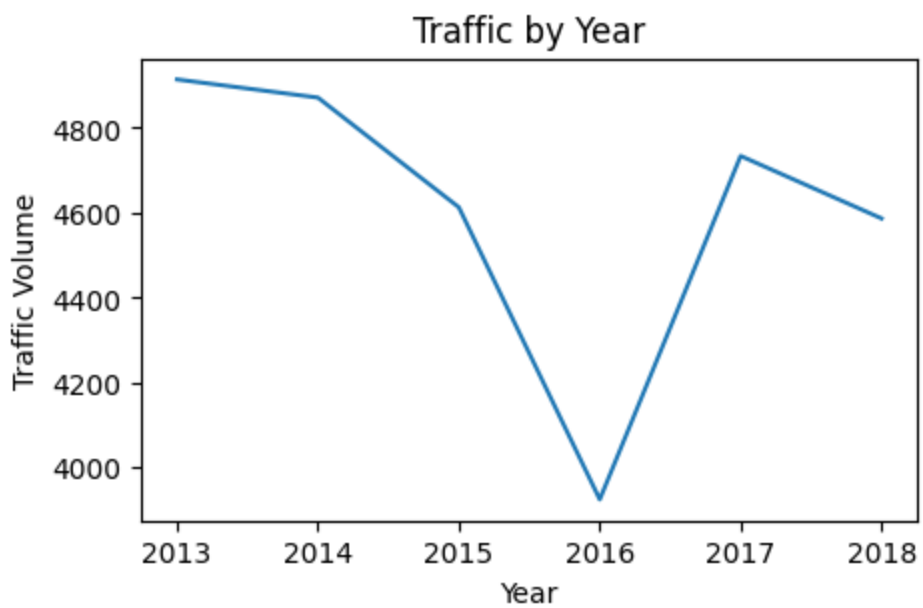
Time may influence traffic volume, so we'll analyze how traffic changes across different time indicators like month, day of the week, and time of day using line plots to identify patterns of heavy traffic.

In [87]:
```
day['month'] = day['date_time'].dt.month
by_month = day.groupby('month').mean(numeric_only=True)
plt.figure(figsize=(5,3))
plt.plot(by_month['traffic_volume'], marker='o')
plt.title('Traffic Volume by Month')
plt.xlabel('Month')
plt.show()
```



The traffic looks less heavy during cold months (November–February) and more intense during warm months (March–October), with one interesting exception: July. Is there anything special about July? Is traffic significantly less heavy in July each year?

In [85]:
```
day['year'] = day['date_time'].dt.year
only_july = day[day['month'] == 7]
plt.figure(figsize=(5,3))
only_july.groupby('year').mean(numeric_only=True)['traffic_volume'].plot.line()
plt.title('Traffic by Year')
plt.xlabel('Year')
plt.ylabel('Traffic Volume')
plt.show()
```

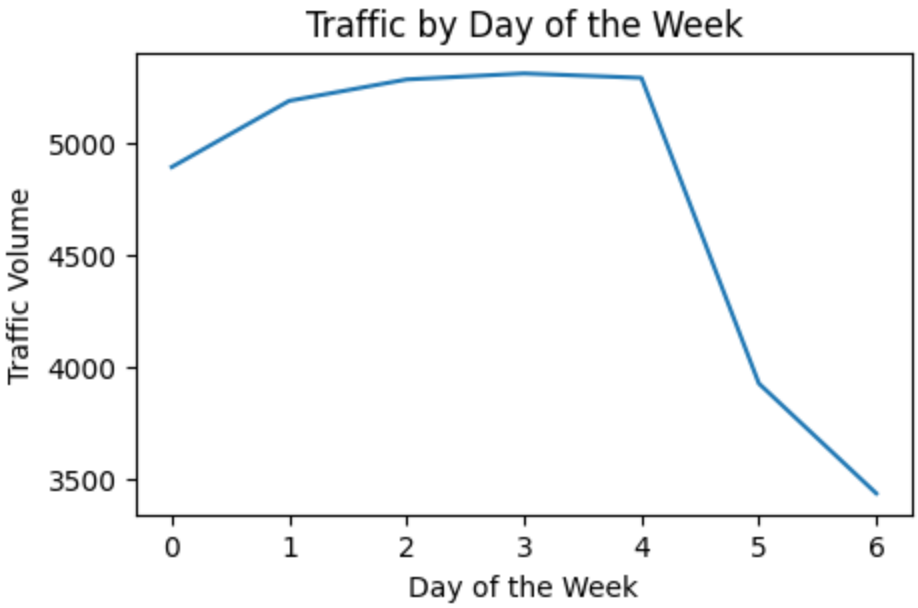


Traffic is generally heavier during warm months, with daytime hourly volumes often approaching 5,000 cars. An exception occurred in July 2016, likely due to road construction,

which caused a noticeable drop.

Analyze traffic changes across Day of the week

In [86]:
```
day['dayofweek'] = day['date_time'].dt.dayofweek
by_dayofweek = day.groupby('dayofweek').mean(numeric_only=True)
plt.figure(figsize=(5,3))
plt.plot(by_dayofweek['traffic_volume'])
days = ['Mon', 'Tue', 'Wed', 'Thurs', 'Fri', 'Sat', 'Sun']
plt.title('Traffic by Day of the Week')
plt.xlabel('Day of the Week')
plt.ylabel('Traffic Volume')
#by_dayofweek['traffic_volume'].plot.line()
plt.show()
```

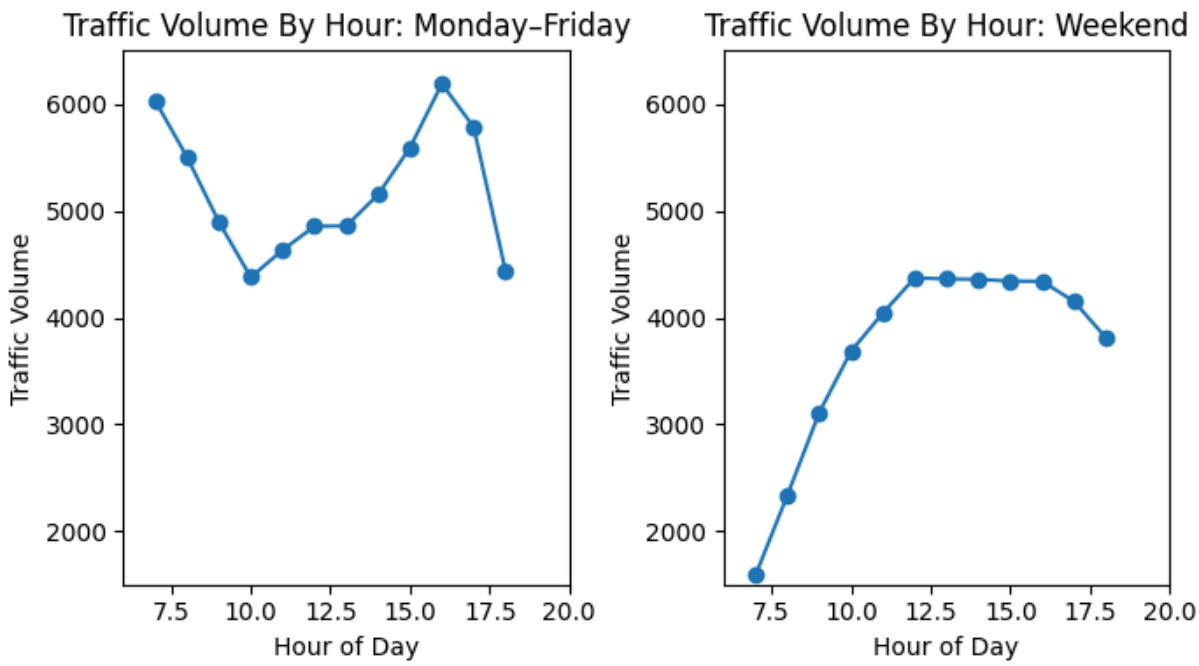


Traffic is much heavier on weekdays (Monday–Friday), often exceeding 5,000 cars per hour, while weekends see lighter traffic, generally below 4,000 cars per hour.

Analyze how traffic changes across Time of the day

In [96]:
```
day['date_time'] = pd.to_datetime(day['date_time'])
day['hour'] = day['date_time'].dt.hour
day['dayofweek'] = day['date_time'].dt.dayofweek

# Separate weekdays and weekends
business_days = day[day['dayofweek'] <= 4]  # Mon-Fri
weekend = day[day['dayofweek'] >= 5]        # Sat-Sun

# Grouping by hour
by_hour_business = business_days.groupby('hour')['traffic_volume'].mean()
by_hour_weekend = weekend.groupby('hour')['traffic_volume'].mean()

# Plot
```

```
plt.figure(figsize=(7,4))

plt.subplot(1, 2, 1)
plt.plot(by_hour_business, marker='o')
plt.xlim(6,20) # to account for differences in 2 graphs
plt.ylim(1500,6500) # to account for differences in 2 graphs
plt.title('Traffic Volume By Hour: Monday-Friday')
plt.xlabel('Hour of Day')
plt.ylabel('Traffic Volume')

plt.subplot(1, 2, 2)
plt.plot(by_hour_weekend, marker='o')
plt.xlim(6,20)
plt.ylim(1500,6500)
plt.title('Traffic Volume By Hour: Weekend')
plt.xlabel('Hour of Day')
plt.ylabel('Traffic Volume')

plt.tight_layout()
plt.show()
```



Traffic volume is generally higher on business days than weekends, with rush hours around 7 AM and 4 PM exceeding 6,000 cars per hour. Overall, heavy traffic tends to occur during warm months (March–October) and on weekdays, particularly during these peak hours.

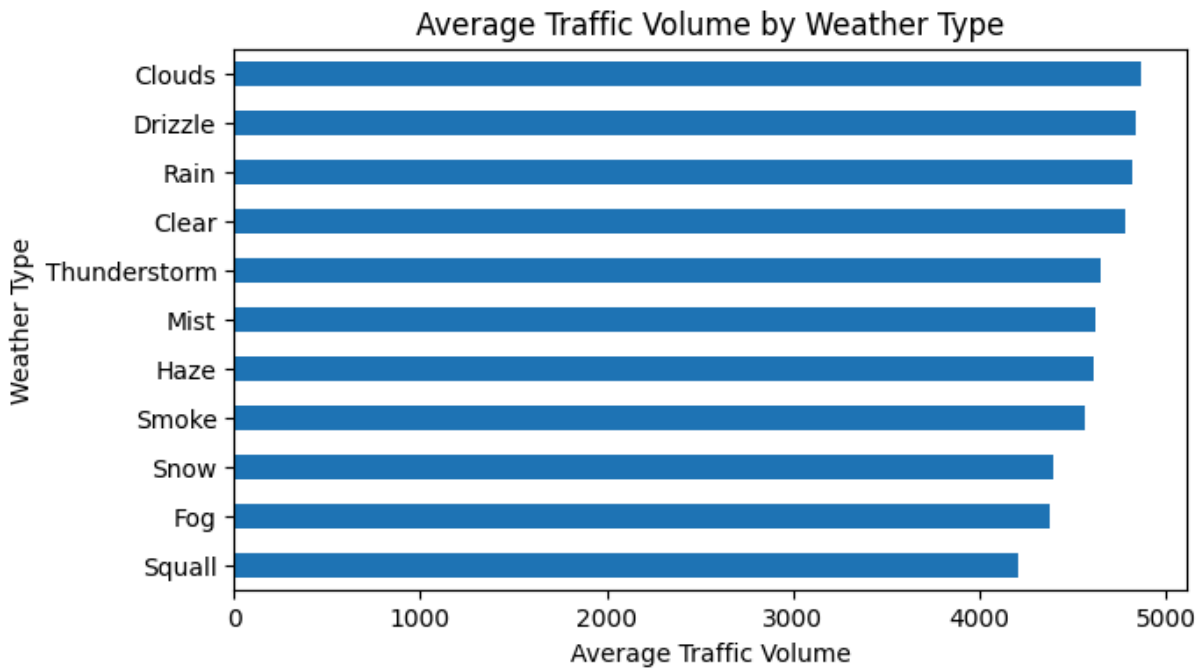
Analyze by Weather

The dataset provides us with a few useful columns about weather: temp, rain_1h, snow_1h, clouds_all, weather_main, weather_description.

In [118…

```
# Analyzing between weather_main & traffic_volume
weather_main = day.groupby('weather_main').mean(numeric_only=True)
```

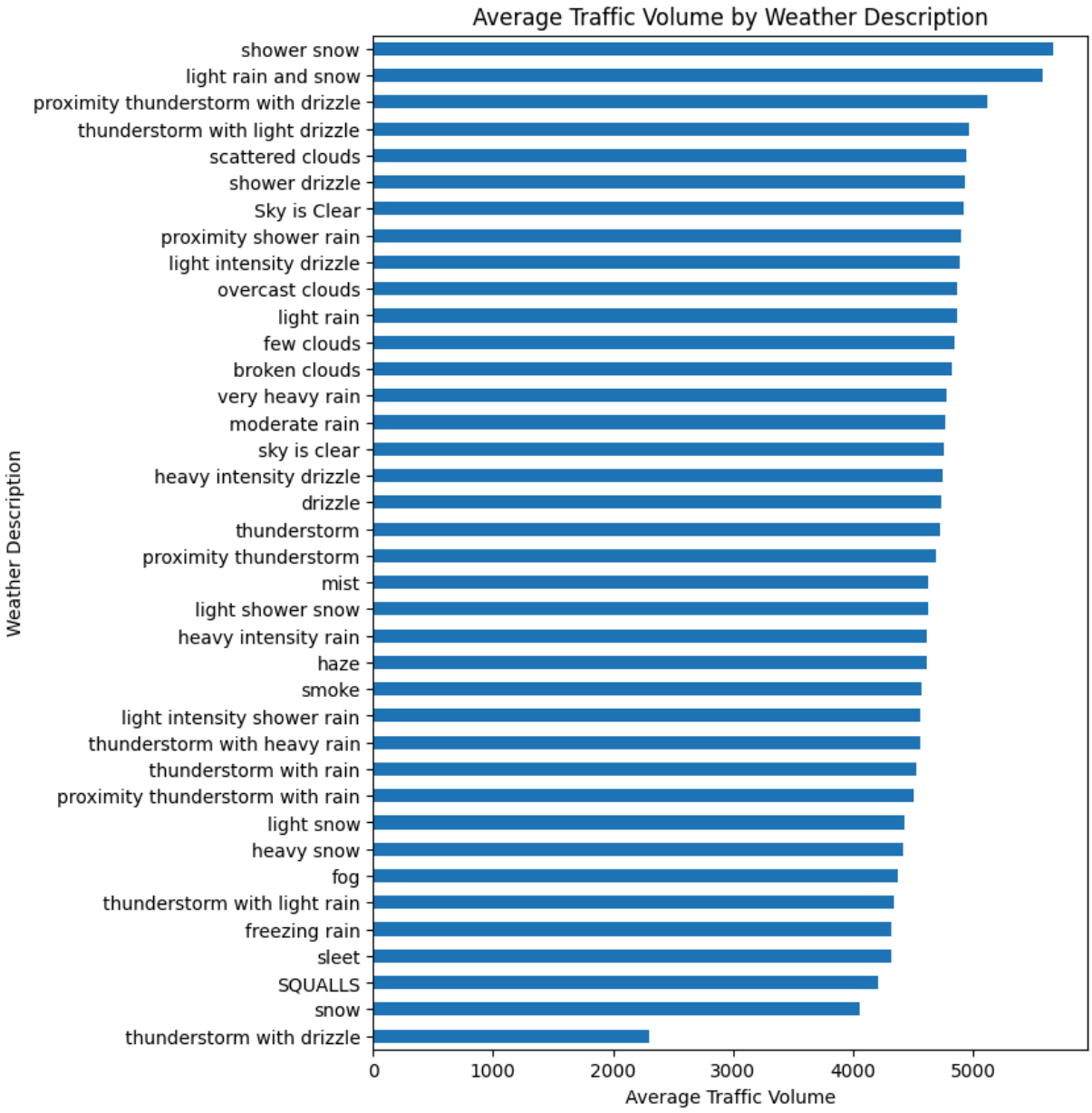
```
plt.figure(figsize=(7,4))
weather_main['traffic_volume'].sort_values(ascending=True).plot.barh()
plt.title('Average Traffic Volume by Weather Type')
plt.xlabel('Average Traffic Volume')
plt.ylabel('Weather Type')
plt.show()
```



No single weather type consistently shows traffic volumes over 5,000 cars, making it a weak indicator of heavy traffic. To gain more insight, we will group by the more detailed weather_description column for a finer analysis.

In [120…

```
# Group by detailed weather description
by_weather_description = day.groupby('weather_description').mean(numeric_only=True)

# Plot
plt.figure(figsize=(7,10))  # taller figure for readability
by_weather_description['traffic_volume'].sort_values(ascending=True).plot.barh()
plt.title('Average Traffic Volume by Weather Description')
plt.xlabel('Average Traffic Volume')
plt.ylabel('Weather Description')
plt.show()
```



In [121…

```
day.groupby('weather_description').count().sort_values('traffic_volume')['dayofweek
```

Out[121…

```
weather_description
SQUALLS                                   1
shower snow                               1
freezing rain                             2
thunderstorm with drizzle                 2
sleet                                     3
shower drizzle                            3
light rain and snow                       4
proximity thunderstorm with drizzle       6
thunderstorm with light drizzle           6
very heavy rain                           7
light intensity shower rain              10
light shower snow                        11
smoke                                    12
proximity thunderstorm with rain         18
thunderstorm with rain                   20
thunderstorm with light rain             23
thunderstorm with heavy rain             25
heavy intensity drizzle                  29
thunderstorm                             48
proximity shower rain                   111
snow                                    137
heavy intensity rain                    219
drizzle                                 278
proximity thunderstorm                  303
heavy snow                              320
fog                                     362
light intensity drizzle                 573
moderate rain                           785
haze                                    834
Sky is Clear                            852
light snow                              978
few clouds                             1143
light rain                             1707
scattered clouds                       2181
overcast clouds                        2585
mist                                   2593
broken clouds                          2793
sky is clear                           4892
Name: dayofweek, dtype: int64
```

Three weather types: “Shower snow,” “Light rain and snow,” and “Proximity thunderstorm with drizzle” — show traffic volumes exceeding 5,000 cars. While these are not extreme conditions, the higher traffic may indicate that people prefer driving rather than walking or biking during moderately bad weather.

In [105…

```
traffic_i94.columns
```

Out[105…

```
Index(['holiday', 'temp', 'rain_1h', 'snow_1h', 'clouds_all', 'weather_main',
       'weather_description', 'date_time', 'traffic_volume'],
      dtype='object')
```

In [106…

```
weather_cols =['temp', 'snow_1h', 'rain_1h', 'clouds_all', 'traffic_volume']
day[weather_cols].corr(numeric_only = 'True')['traffic_volume'].sort_values() # ide
```

Out[106…

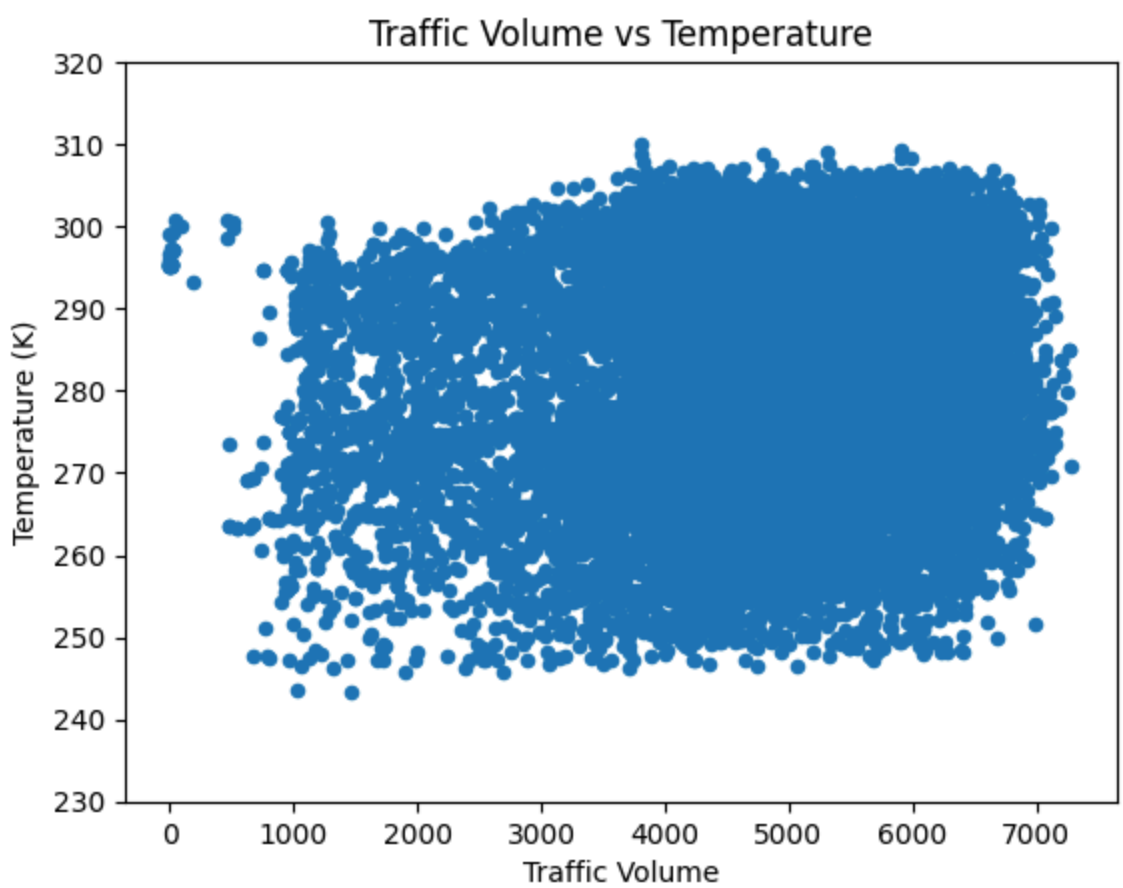
```
clouds_all       -0.032932
snow_1h           0.001265
rain_1h           0.003697
temp              0.128317
traffic_volume    1.000000
Name: traffic_volume, dtype: float64
```

Temperature shows the strongest but still weak—correlation with traffic volume (+0.128). Other weather-related variables like rain_1h, snow_1h, and clouds_all have little to no correlation, suggesting that weather is generally not a strong predictor of traffic volume.

In [111…

```
# correlation between temp and traffic_volume
plt.figure(figsize=(5,4))
day.plot.scatter('traffic_volume', 'temp')
plt.ylim(230, 320)  # remove the extreme 0K temperatures
plt.title('Traffic Volume vs Temperature')
plt.xlabel('Traffic Volume')
plt.ylabel('Temperature (K)')
plt.show()
```

```
<Figure size 500x400 with 0 Axes>
```

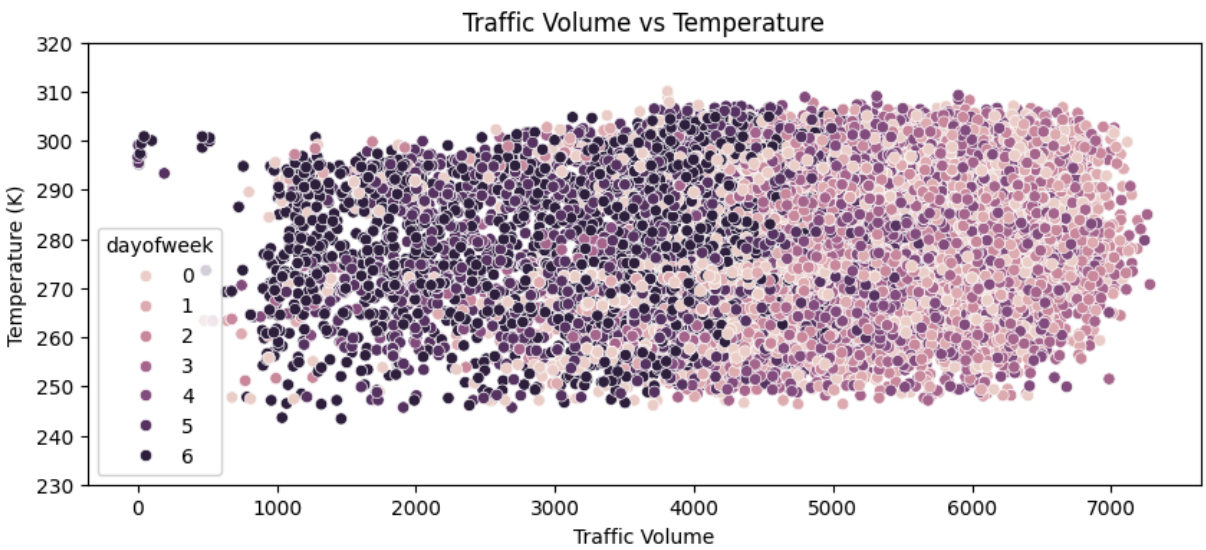


Hence we can conclude that temperature doesn't look like a solid indicator of heavy traffic.

In [114…

```
# correlation between temp and traffic_volume
plt.figure(figsize=(10,4))
sns.scatterplot(x='traffic_volume', y='temp', hue ='dayofweek', data=day)
plt.ylim(230,320)
```

```
plt.title('Traffic Volume vs Temperature')
plt.xlabel('Traffic Volume')
plt.ylabel('Temperature (K)')
plt.show()
```



Weekends have less strict traffic!

This analysis identified two key indicators of heavy traffic on the I-94 Interstate:

Time indicators: Traffic is heavier during warm months (March–October), on weekdays compared to weekends, and peaks around 7 AM and 4 PM on business days. Weather indicators: Higher traffic volumes occur during “Shower snow,” “Light rain and snow,” and “Proximity thunderstorm with drizzle.”

How holidays affect Traffic

In [124…

```
day.groupby('holiday')['traffic_volume'].mean()
```

Out[124…

```
Series([], Name: traffic_volume, dtype: float64)
```

In [126…

```
print(day['holiday'].unique())
print(day['holiday'].isnull().sum())
```

```
[nan]
23877
```

In [127…

```
# Replacing NAN as string
day['holiday'] = day['holiday'].astype(str)
```

In [128…

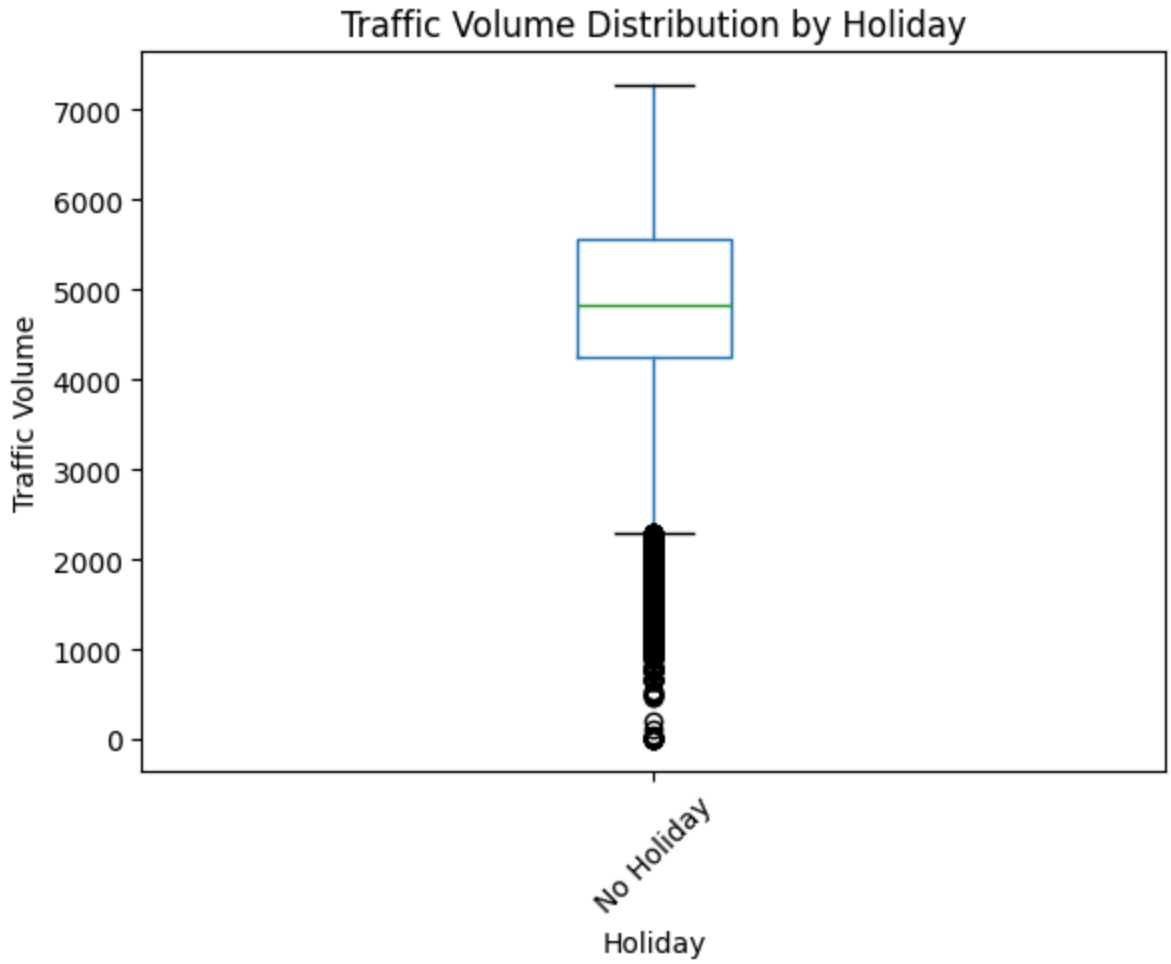
```
day['holiday'] = day['holiday'].replace('nan', 'No Holiday')
```

In [132…

```
# Boxplot showing the distribution of traffic volumes for holidays versus non-holid

# Fill missing holidays with 'No Holiday'
day['holiday'] = day['holiday'].fillna('No Holiday')
```

```
plt.figure(figsize=(10,5))
day.boxplot(column='traffic_volume', by='holiday', grid=False)
plt.title('Traffic Volume Distribution by Holiday')
plt.suptitle('')  # remove default pandas title
plt.xlabel('Holiday')
plt.ylabel('Traffic Volume')
plt.xticks(rotation=45)
plt.show()
```

```
<Figure size 1000x500 with 0 Axes>
```



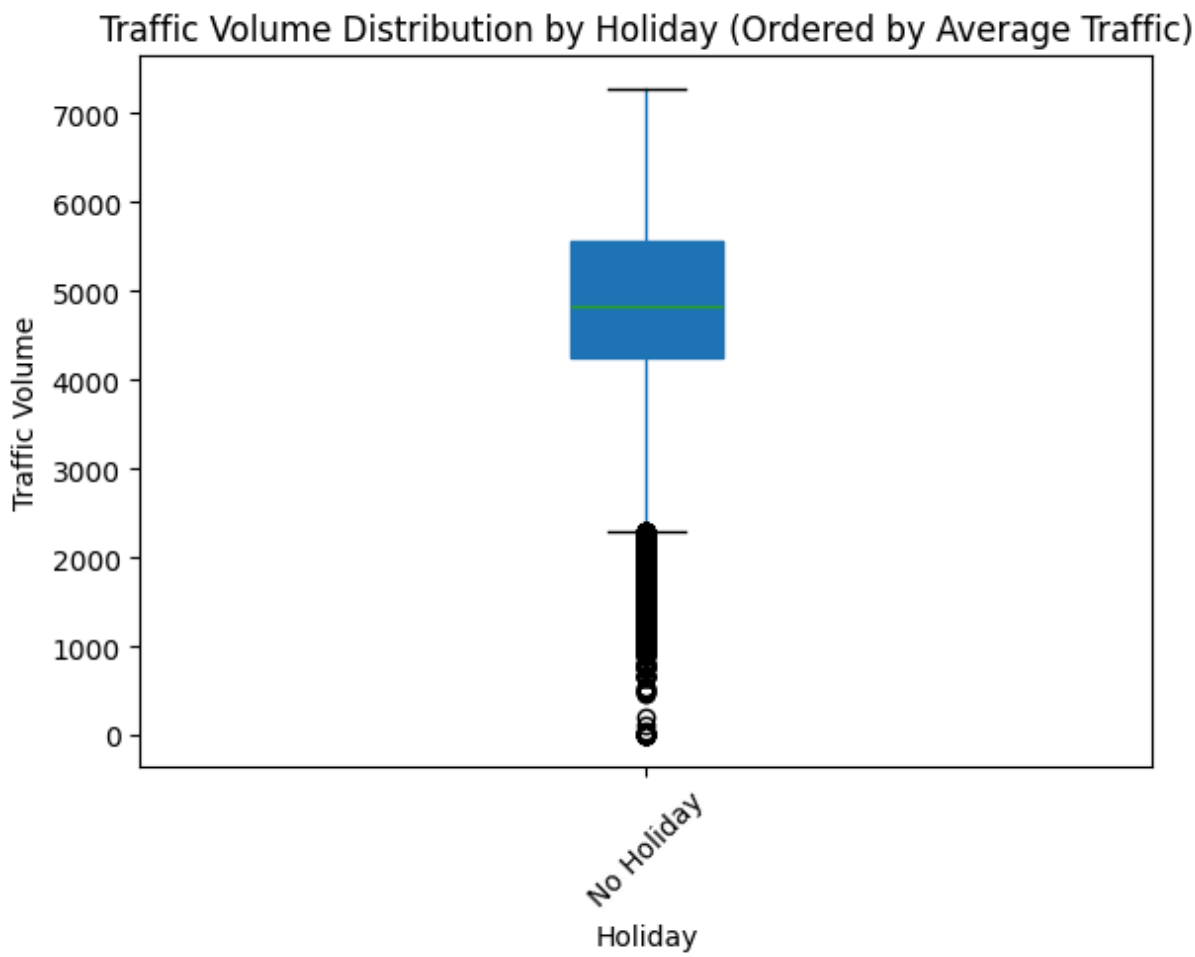
The dataset currently contains only non-holiday data. Median traffic is about 4,800 cars/hour, with most values between 4,200 and 5,500. Traffic generally ranges from 2,200 to 7,200 cars/hour, and very low volumes (0–2,200) are outliers, likely occurring late at night or early morning.

In [133…

```
# distribution of traffic volume on days with No Holiday.

# Compute average traffic per holiday
avg_traffic = day.groupby('holiday')['traffic_volume'].mean().sort_values(ascending

# Order the holiday column by avg traffic
ordered_holidays = avg_traffic.index.tolist()

# Plot boxplot with holidays in descending traffic order
plt.figure(figsize=(12,6))
```

```
day.boxplot(column='traffic_volume', by='holiday', grid=False, positions=range(len(

# Set x-ticks and labels
plt.xticks(ticks=range(len(ordered_holidays)), labels=ordered_holidays, rotation=45
plt.title('Traffic Volume Distribution by Holiday (Ordered by Average Traffic)')
plt.suptitle('')  # remove default pandas title
plt.xlabel('Holiday')
plt.ylabel('Traffic Volume')
plt.show()
```

```
<Figure size 1200x600 with 0 Axes>
```



Median traffic (~4,800–5,000): The typical traffic volume on non-holiday days is fairly high and centered around this range. Interquartile range (IQR ~4,200 to ~5,600): Most traffic values fall within this band, indicating moderate variability during normal days. Upper range (~7,200+): On busy days, traffic can get quite high, suggesting peak commuting or heavy usage periods. Lower range (~2,300): The lower whisker shows that even on quieter non-holiday days, traffic generally stays above ~2,300. Many low outliers (0–2,000): There are numerous unusually low traffic points. These likely represent: Late-night or early-morning periods or Unusual disruptions (weather, incidents, etc.) Overall Interpretation: 1. Traffic on non-holidays is consistently high but somewhat variable. 2. The presence of many low outliers suggests occasional sharp drops, but these are not typical. 3. The distribution is slightly skewed downward due to those low outliers.

Effect of rainfall and snow on traffic

In [134…

```
day[['traffic_volume', 'rain_1h', 'snow_1h']].corr()
```

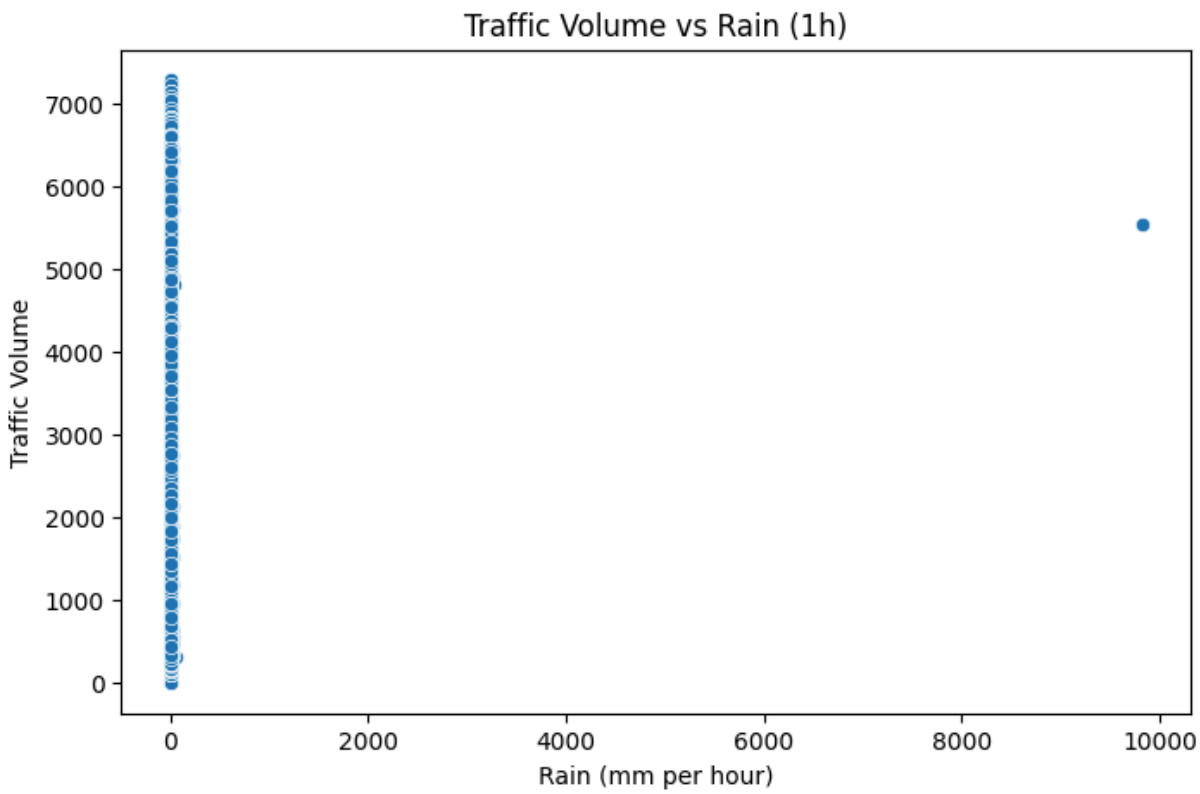
Out[134…

	traffic_volume	rain_1h	snow_1h
traffic_volume	1.000000	0.003697	0.001265
rain_1h	0.003697	1.000000	-0.000091
snow_1h	0.001265	-0.000091	1.000000

Because both correlation coefficients are nearly zero, it's concluded that rainfall and snowfall have no linear relationship with traffic volume in this dataset.

In [136…

```
plt.figure(figsize=(8,5))
sns.scatterplot(data=traffic_i94, x='rain_1h', y='traffic_volume')
plt.title("Traffic Volume vs Rain (1h)")
plt.xlabel("Rain (mm per hour)")
plt.ylabel("Traffic Volume")
plt.show()
```



Most of the data points are at 0 for rain, showing it has little observable effect on traffic volume in this dataset.

In [139…

```
# Creating a new column for combined events
traffic_i94['combined_weather'] = traffic_i94.apply(lambda row:
                                    'rain+snow' if row['rain_1h'] > 0 and row['snow_1
                                    else 'rain_only' if row['rain_1h'] > 0
                                    else 'snow_only' if row['snow_1h'] > 0
                                    else 'other', axis=1)
```

```python
# Quick check
traffic_i94['combined_weather'].value_counts()
```

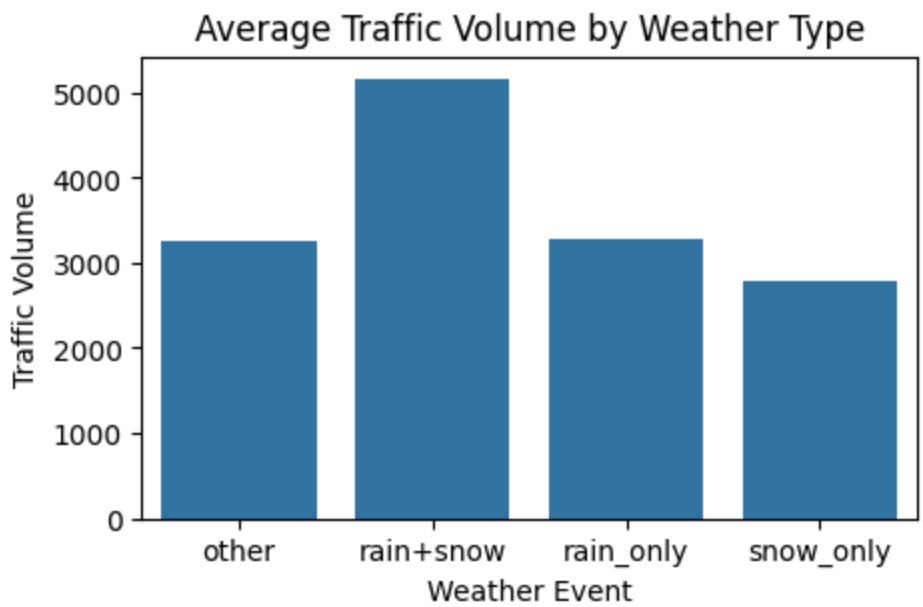
Out[139…

```
combined_weather
other        44678
rain_only     3463
snow_only       59
rain+snow        4
Name: count, dtype: int64
```

In [141…

```python
# Grouping by combined_weather and compute mean traffic
traffic_comparison = traffic_i94.groupby('combined_weather')['traffic_volume'].mean
print(traffic_comparison)

plt.figure(figsize=(5,3))
sns.barplot(data=traffic_comparison, x='combined_weather', y='traffic_volume')
plt.title('Average Traffic Volume by Weather Type')
plt.ylabel('Traffic Volume')
plt.xlabel('Weather Event')
plt.show()
```

```
  combined_weather  traffic_volume
0            other     3257.956511
1        rain+snow     5167.000000
2        rain_only     3289.679757
3        snow_only     2787.694915
```



Rain + snow has the highest traffic volume (around 5,100). This suggests that even in more severe mixed weather, traffic remains high possibly due to people still need to travel despite conditions. Rain only and other weather conditions are moderate and similar (around 3,200–3,300), indicating typical traffic levels. Snow only has the lowest traffic volume (around 2,800), implying that snow alone discourages travel more than rain or mixed conditions.

Overall insight: Traffic tends to drop in snowy conditions, but surprisingly peaks when rain and snow occur together, which could indicate timing (e.g., rush hours) or limited alternatives for commuters.

In [144…

```
# Looking for rare or suspicious weather descriptions
rare_events = traffic_i94['weather_description'].value_counts().loc[lambda x: x < 5
print("Rare/suspicious events:\n", rare_events)
```

```
Rare/suspicious events:
 weather_description
SQUALLS                       4
sleet                         3
freezing rain                 2
thunderstorm with drizzle     2
shower snow                   1
Name: count, dtype: int64
```

In [147…

```
from datetime import datetime

# Define construction periods (example dates based on DOT info)
construction_periods = [
    ("2025-11-01", "2026-12-31"),  # West Leg I-94 impacts begin Nov 2025
]

# Convert to datetime
construction_periods = [
    (datetime.fromisoformat(start), datetime.fromisoformat(end))
    for start, end in construction_periods
]
```

In [149…

```
# Ensure your date column is datetime
traffic_i94['date_time'] = pd.to_datetime(traffic_i94['date_time'])

# Flag rows in construction period
traffic_i94['is_construction'] = traffic_i94['date_time'].apply(
    lambda dt: any(start <= dt <= end for start, end in construction_periods)
)
```

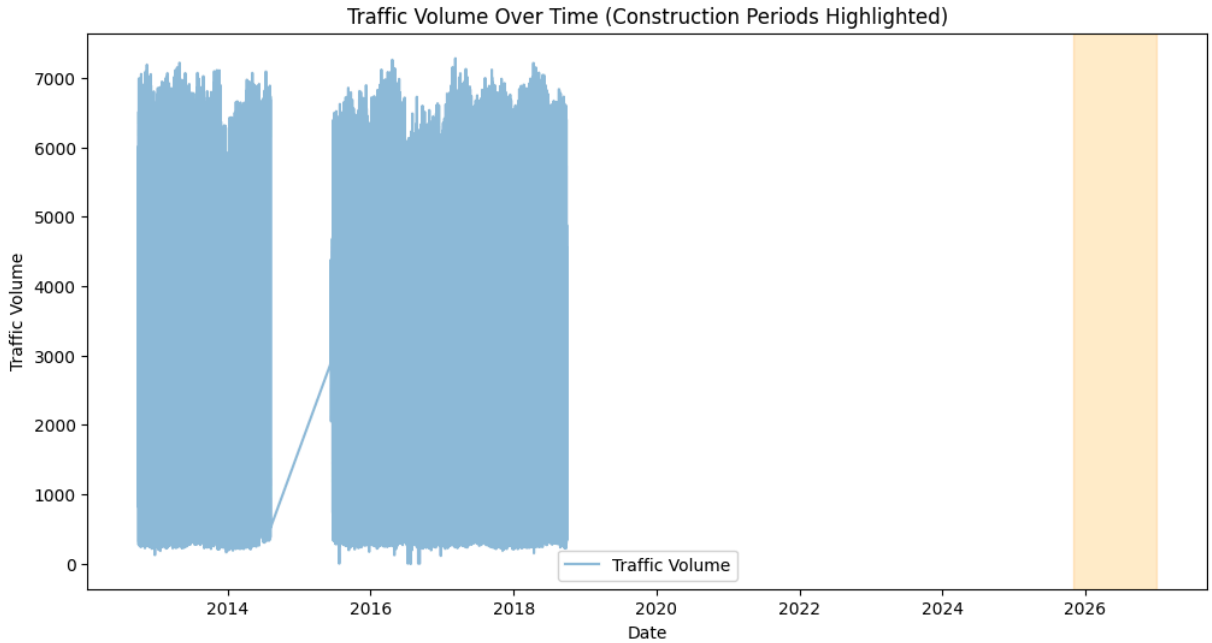
In [150…

```
traffic_by_period = traffic_i94.groupby('is_construction')['traffic_volume'].agg(['
print(traffic_by_period)
```

```
                       mean  median  count
is_construction
False            3259.818355  3380.0  48204
```

In [151…

```
plt.figure(figsize=(12,6))
plt.plot(traffic_i94['date_time'], traffic_i94['traffic_volume'], label='Traffic Vo

# Shade construction intervals
for start, end in construction_periods:
    plt.axvspan(start, end, color='orange', alpha=0.2)

plt.title("Traffic Volume Over Time (Construction Periods Highlighted)")
plt.xlabel("Date")
plt.ylabel("Traffic Volume")
plt.legend()
plt.show()
```



Key observations: 1. Consistently high traffic (2013–2018): Traffic volume generally fluctuates between ~6,000 and 7,200, indicating steady and heavy usage during these years. 2. Regular variability: There are frequent ups and downs, likely reflecting daily or seasonal patterns (e.g., rush hours, weekdays vs. weekends). 3. Sudden dips/spikes: Occasional sharp drops (near zero) and quick recoveries suggest anomalies, possibly due to missing data, holidays, or temporary disruptions. 4. Gap or missing data after ~2018: There's a large empty space between ~2019 and 2025, implying no recorded data or it's not displayed. 5. Construction period (highlighted around 2026–2027): This section is shaded but shows no visible traffic data, so you can't directly compare traffic before vs. during construction from this chart alone. Overall insight: Traffic was stable and high before 2019, but due to missing data in later years—including the construction period it's not possible to assess the impact of construction on traffic volume from this visualization alone.

In [153…

```
from nbconvert import HTMLExporter
from nbformat import read
import webbrowser

nb_path = "I94_Traffic.ipynb"
out_path = "I94_Traffic.html"

with open(nb_path, "r", encoding="utf-8") as f:
    nb = read(f, as_version=4)

exporter = HTMLExporter()
body, _ = exporter.from_notebook_node(nb)

with open(out_path, "w", encoding="utf-8") as f:
    f.write(body)

webbrowser.open(out_path)
print("opened:", out_path)
```

```
opened: I94_Traffic.html
```

In [ ]: